AI Interaction Log

CIE-1, Computer Organisation & Operating Systems | TE IT | Roll No. __ , Name _____ | Assistant used: Claude (Opus 5), free tier, browser

I used AI for four things during this assignment: understanding the free/available distinction, predicting the locality result before measuring it, reading the ps STAT codes, and debugging my C timing loop. Two of those exchanges contained errors that my own machine disproved. Both are recorded below with the command output that settled them.

Entry 1	AI WAS WRONG
My prompt	How much memory does my machine actually have available right now, and what happens if a program asks for more than that?
AI answer	It said that used memory is total minus free, so with 3.9 GiB total and 3.1 GiB free only about 800 MiB was in use, and that roughly 3.1 GiB was available to a new process. It added that if I allocated more than that, the system would start swapping to disk and slow down but keep running.
How I checked	I ran free -h and read the last two columns, then checked the Swap row, then read MemAvailable in /proc/meminfo to confirm the number was the kernel's and not a calculation free made up.

Evidence	`$ free -h`
	`total used free shared buff/cache available`
	`Mem: 3.9Gi 287Mi 3.1Gi 5.0Mi 709Mi 3.6Gi`
	`Swap: 0B 0B 0B`

Conclusion	Two errors. First, available is 3.6 GiB, not 3.1 GiB, because 709 MiB of page cache is reclaimable and the kernel counts it as available. The AI's subtraction double-counted cache as unavailable. Second, and more seriously, Swap is 0 B on this machine. There is nothing to swap to. An over-allocation here does not degrade gracefully; the OOM killer terminates a process. The AI gave a confident answer about swap behaviour without knowing whether swap existed, and it had no way to know, because I had not told it.

Entry 2	AI WAS OVERCONFIDENT
My prompt	Before I run it: how much slower should column-major traversal of an 8192x8192 double array be, compared with row-major, on a normal x86 machine?
AI answer	It reasoned that a 64-byte cache line holds 8 doubles, so row-major order gets 8 useful accesses per line fetched while column-major gets 1, and predicted a slowdown of roughly 8x, possibly more once TLB misses are counted.
How I checked	I ran the experiment three times and compared the mean against the prediction.

Evidence	`run,row_major_s,col_major_s,ratio`
	`1,0.1736,0.6267,3.61`
	`2,0.1798,0.6225,3.46`
	`3,0.1778,0.6470,3.64`
	`mean,0.1771,0.6321,`**`3.57`**

Conclusion	The direction was right and the mechanism was right, but the magnitude was out by more than a factor of two. Measured 3.57x, predicted 8x or more. The cache-line argument gives an upper bound, not a prediction: it assumes every column access pays full memory latency. In practice the column stride is large but perfectly regular, so the hardware prefetcher can follow it and hide part of the cost. The AI presented an upper bound as an expected value. I would not have caught this without measuring.

Entry 3	AI ASSUMED WRONG HARDWARE
My prompt	In htop, how do I read the per-core CPU bars to see if my load is spread evenly across cores?
AI answer	It described reading the stack of coloured bars at the top left, one per logical core, and suggested watching for a single core pinned at 100% while others idle as a sign of a single-threaded bottleneck. It assumed a typical 4- to 8-core laptop with hyperthreading enabled.
How I checked	I checked what lscpu actually reports for this machine before following the advice.

Evidence	`$ lscpu | grep -E 'CPU\(s\)|Thread|Core'`
	`CPU(s):  1`
	`Thread(s) per core:  1`
	`Core(s) per socket:  1`

Conclusion	There is one logical CPU and no hyperthreading, so there is exactly one bar and the whole per-core comparison is meaningless on this machine. The advice was not wrong in general, it was wrong here. This is the failure mode I now watch for: the AI answers for a typical machine because it cannot see mine, and a plausible answer about the wrong hardware reads exactly like a correct one.

Entry 4	AI WAS CORRECT
My prompt	What do the STAT letters mean in ps output, specifically I and D?
AI answer	It said I is an idle kernel thread, D is uninterruptible sleep normally caused by blocking I/O, and that a D-state process cannot be killed with SIGKILL until the operation completes.
How I checked	Cross-checked against the PROCESS STATE CODES section of man ps, and confirmed the I rows in my own output were all kworker kernel threads.

Evidence	`$ ps -eo pid,stat,comm | grep ' I ' | head -3`
	`  11 I  kworker/0:1-virtio_vsock`
	`  12 I  kworker/u4:0-events_unbound`

Conclusion	Accurate and matched the manual page. I used this wording, in my own words, for the state column of the dashboard.

What I take from this

The pattern in entries 1 to 3 is the same. The AI was fluent about memory, caches and cores in general, and wrong about this machine in particular, because it could not see this machine and I had not given it the output. Its confidence did not drop when its knowledge did. The one case where it was reliable, entry 4, was a question about documented meanings rather than about my hardware, which is the kind of question it should be good at.

The practical rule I ended up with: use it to explain what a column means, never to tell me what the value in that column is. Everything in Report.pdf that is a number came from a command; everything that is an explanation was checked against the manual page or against a measurement.